



Security Audit

SPICE “Eigen” & “Swappers” (DeFi)

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	7
Security Rating	8
Intended Smart Contract Functions	9
Code Quality	10
Audit Resources	10
Dependencies	10
Severity Definitions	11
Status Definitions	12
Audit Findings	13
Centralisation	37
Conclusion	38
Our Methodology	39
Disclaimers	41
About Hashlock	42

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The Spice team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

SPICE provides risk coverage infrastructure for secured lending to real-world businesses. A standardized coverage layer, backed by staked assets, delivers instant payouts when risk events occur, making institutional credit investable on-chain.

Project Name: Spice

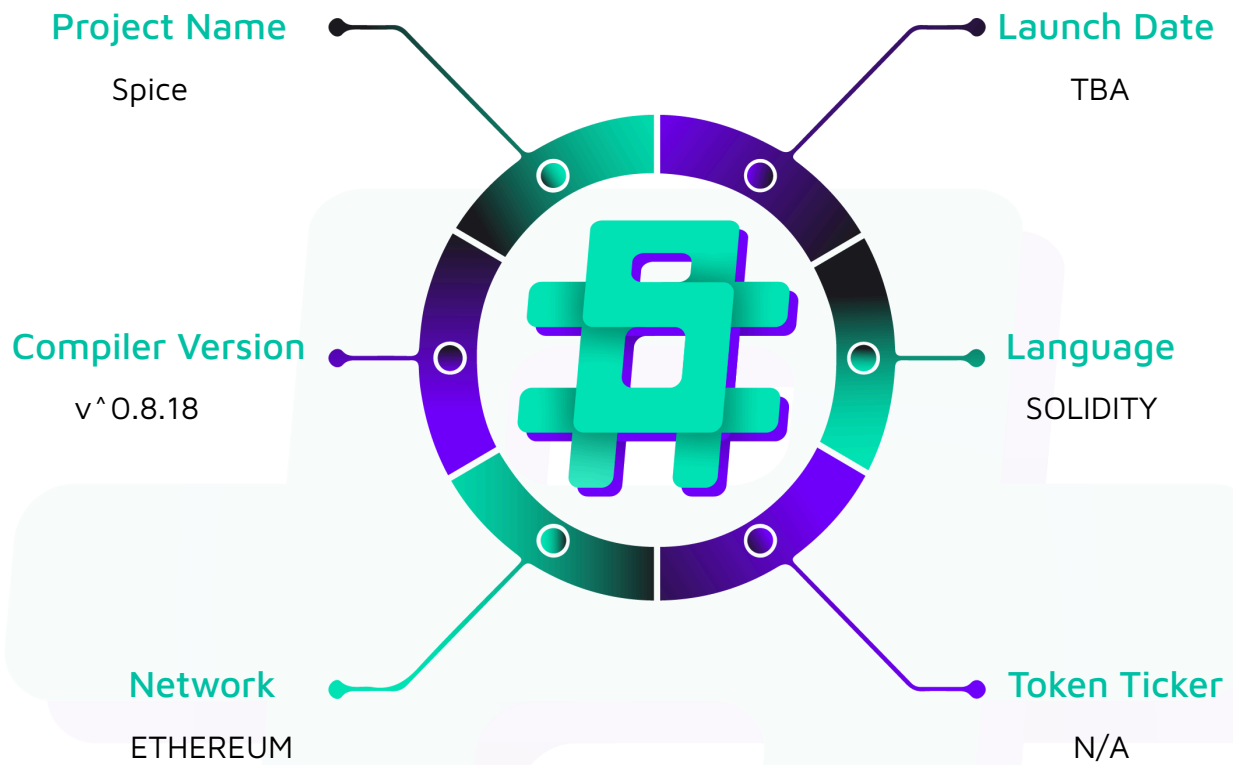
Project Type: DeFi

Compiler Version: ^0.8.18

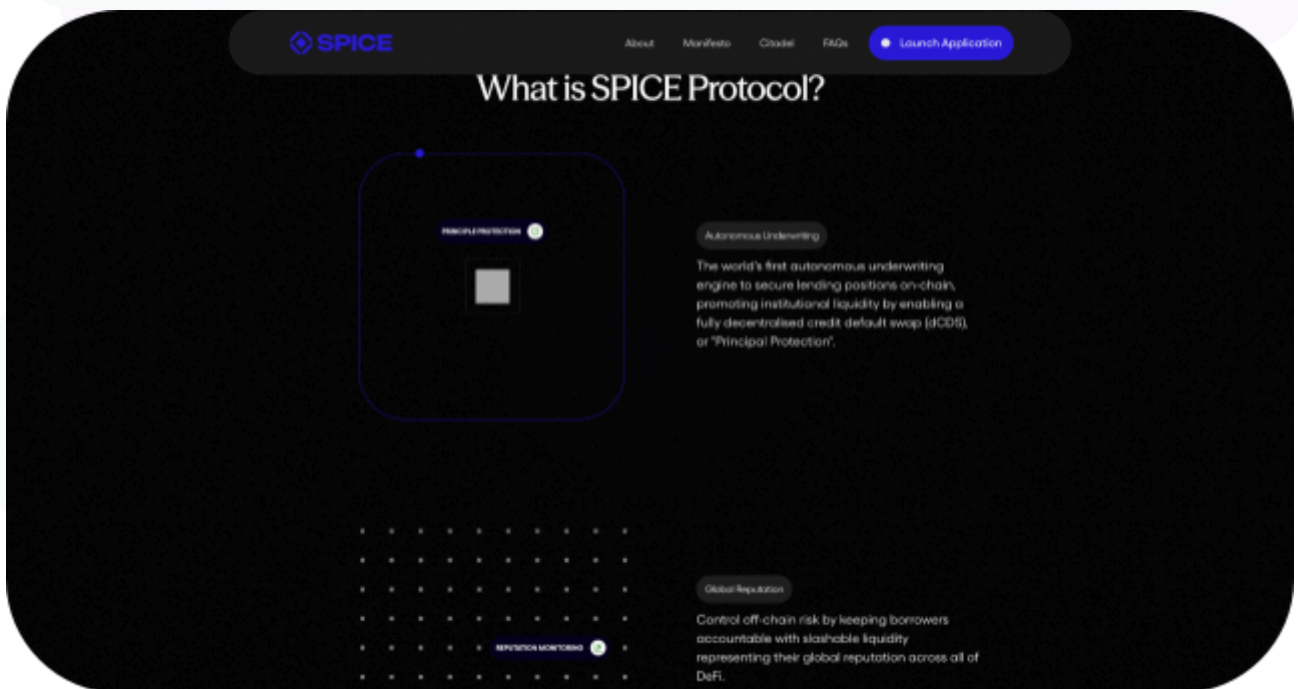
Website: <https://spiceprotocol.xyz/>

Logo:



Visualised Context:

Project Visuals:



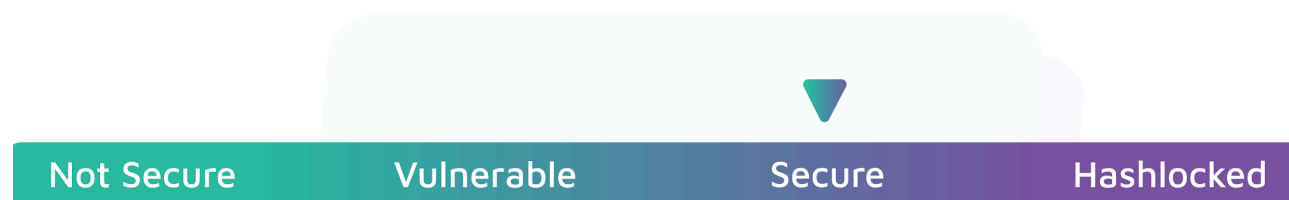
Audit Scope

We at Hashlock audited the solidity code within the Spice project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	Spice Smart Contracts
Network	Ethereum
Language	Solidity
Audit Date	February, 2026
Contract 1	EigenCoverageProviderFacet.sol
Contract 2	EigenServiceManagerFacet.sol
Contract 3	UniswapV3SwapperEngine.sol
Contract 4	AssetPriceOracleAndSwapper.sol
Audited GitHub Commit Hash	9cf4122f346533671eb431a5eaadda2257811ad9
Fix Review GitHub Commit Hash	1d15cc8804646e78058094f8736a6361c1429e35

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.



The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

3 High severity vulnerabilities

7 Medium severity vulnerabilities

2 Low severity vulnerabilities

3 QAs

Caution: Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
EigenCoverageProviderFacet.sol - Manages coverage positions and claims on EigenLayer—accepts/reserves/issues claims, holds/streams rewards, enforces slashing and liquidations, and coordinates allocations and rewards with EigenLayer components.	Contract achieves this functionality.
EigenServiceManagerFacet.sol - Integrates with EigenLayer operator/strategy sets—manages operator registration, strategy whitelisting, allocation checks, operator-directed reward submissions, and slashing interactions for coverage bookkeeping.	Contract achieves this functionality.
UniswapV3SwapperEngine.sol - Delegatecallable swapper engine using Uniswap V3 Universal Router and Permit2 to quote and execute exact-in/exact-out swaps (with path handling and quoter support) for modular swapping.	Contract achieves this functionality.
AssetPriceOracleAndSwapper.sol - Mixin that registers asset pairs, selects between oracle or swapper pricing strategies, verifies quotes, and performs delegated swaps with configurable slippage and pool initialization.	Contract achieves this functionality.

Code Quality

This audit scope involves the smart contracts of the Spice project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring is recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Spice project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] EigenCoverageProviderFacet#_initiateSlash - Partial Slash
Permanently Locks Unslashed Coverage

Description

When a coverage claim undergoes a partial slash (or a zero-amount slash), the slashed amount is correctly deducted from the operator's active coverage map. However, the claim's status is then permanently updated to Slashed, which leaves the unslashed remainder of the coverage amount locked indefinitely with no path for cleanup.

Vulnerability Details

The `_initiateSlash` function reduces the coverage map by the specific amount being slashed rather than the full `_claim.amount`. Following this, it updates the claim's status to `Slashed`. Since the `closeClaim` function strictly requires a claim to be in either an `Issued` or `Reserved` state, it will revert for `Slashed` claims.

Consequently, the unslashed remainder of the claim is never freed from the coverage map. Furthermore, because there is no validation preventing zero-amount slashes, a malicious coverage agent can exploit this by executing a zero-amount slash on a valid claim. This changes the status to `Slashed` without actually slashing any funds, permanently locking the entire allocated claim amount.

Impact

Repeated partial slashes will permanently deplete an operator's coverage capacity, leading to a permanent Denial of Service (DoS). The operator will be unable to serve new coverage agents despite having adequate underlying stake. A malicious coverage agent can weaponize zero-amount slashes to instantly brick an operator's entire capacity at zero cost.

Recommendation

Ensure the full claim amount is released from the coverage map during a slash, rather than just the slashed portion.

```
// Release the FULL claim amount, not just the slashed portion

_modifyCoverageForAgent(operator,          strategy,          _position.coverageAgent,
-int256(_claim.amount));
```

Additionally, explicitly prevent zero-amount slashes by adding a validation check:

```
require(amount > 0, "Slash amount must be positive");
```

Status

Resolved

[H-02] EigenServiceManagerFacet#_totalAllocatedValueToCoverageAgent - iGetAllocatedStake Returns Shares Not Underlying, Breaking Coverage and Slashing Math

Description

The protocol values operator coverage backing and computes slashing proportions from `IAAllocationManager.getAllocatedStake(...)`. This flow assumes the returned amount is underlying tokens.

In EigenLayer, `getAllocatedStake(...)` returns strategy shares, not underlying. For yield-bearing strategies, share value drifts over time. Treating shares as underlying introduces a unit mismatch in both coverage valuation and wadToSlash calculation.

```
// src/providers/eigenlayer/facets/EigenServiceManagerFacet.sol

function _totalAllocatedValueToCoverageAgent(...) internal view returns (...) {
    // ...

    uint256[][] memory allocatedStake = IAAllocationManager(...)
        .getAllocatedStake(operatorSet, _operators, strategies);

    (total, verified) = IAssetPriceOracleAndSwapper(address(this))
        .getQuote(allocatedStake[0][0], coverageAsset, strategyAsset);

    // @audit allocatedStake[0][0] is shares, quoted as underlying
    // ...
}

function _calculateWadToSlash(...) internal view returns (uint256 wadToSlash) {
    // ...

    uint256 totalAllocatedStake = IAAllocationManager(...)
```

```

        .getAllocatedStake(...)[0][0];

        wadToSlash = (requiredSlashAmount * WAD) / totalAllocatedStake;

        // @audit requiredSlashAmount is underlying; denominator is shares
        // ...
    }

```

Impact

Coverage can be materially under- or over-valued, depending on the share-to-underlying ratio. This can block valid claims or permit claims against mispriced backing. Slashing is also miscomputed: operators may be over-slashed (direct loss) or under-slashed (insufficient settlement and downstream reverts), weakening coverage guarantees.

Recommendation

Convert shares to underlying before using allocated stake in pricing and slash math, and apply this consistently in all related code paths.

```

uint256 shares = IAllocationManager(...)

        .getAllocatedStake(operatorSet, _operators, strategies)[0][0];

uint256 underlyingAmount = IStrategy(strategy).sharesToUnderlyingView(shares);

(total, verified) = IAssetPriceOracleAndSwapper(address(this))

        .getQuote(underlyingAmount, coverageAsset, strategyAsset);

wadToSlash = (requiredSlashAmount * WAD) / underlyingAmount;

```

Status

Resolved

[H-03] EigenCoverageProviderFacet#captureRewards - captureRewards Drains Diamond Token Balance via Unfunded Reserved Claims

Description

The reward capture flow assumes that any claim with a non-zero reward is already funded inside the diamond. That assumption is true for issued claims because tokens are transferred during issuance, but it is not true for reserved claims, where no reward tokens are transferred.

Vulnerability Details

`reserveClaim(...)` stores a user-controlled reward value in the claim without transferring tokens into the diamond. Later, `captureRewards(...)` calculates distributable rewards from the claim state without requiring the claim to be in an issued or completed lifecycle state for `Refundable.None`.

Because reserved claims do not initialize distributed reward accounting, `claimRewardDistributions[claimId]` is effectively zero on first capture. This allows:

```
amount = _claim.reward - _claimRewardDistribution.amount;
```

to resolve the full unfunded reserved reward. The contract then approves and submits that amount to `RewardsCoordinator`, pulling from the diamond's current token balance for that asset, including funds deposited by other valid claims.

The same principle applies to `Refundable.TimeWeighted`, where repeated captures can extract the reserved reward over time. `Refundable.Full` is gated by `Completed` status and does not expose the same immediate path.

Impact

An attacker controlling a coverage agent can reserve unfunded claims with inflated reward values and route the diamond's already-funded reward tokens to their own operator reward stream. In practice, this can drain the diamond-held balance of the affected reward token and steal funds intended for legitimate claims.

Recommendation

- Block reward capture for reserved claims by enforcing a strict status gate in `captureRewards(...)`.
- Treat only properly funded claim states as reward-eligible.
- As defense in depth, initialize and validate reward-distribution accounting consistently across all claim states.

Status

Resolved

Medium

[M-01] UniswapV3SwapperEngine#swapForOutput, swapForInput - Spot Pricing and Unbounded Deadlines Expose Protocol Swaps to Sandwich Attacks

Description

The protocol uses momentary spot prices from the Uniswap V3 quoter to derive its slippage protection parameters for protocol-initiated token swaps (e.g., via `swapForOutput` and `swapForInput`), making the swap inherently vulnerable to sandwich attacks. Additionally, the swaps execute with a transaction deadline hardcoded to `block.timestamp`, providing no protection against delayed execution by validators.

Vulnerability Details

In `AssetPriceOracleAndSwapper.sol`, functions like `swapForOutput` calculate the slippage threshold using `swapForOutputQuote`, which delegates to the `UniswapV3SwapperEngine.getQuote()` function. This engine retrieves the exact on-chain spot price at the time of execution by calling `quoteExactInput` on the Uniswap V3 quoter. Since both the price quote calculation and the swap execution occur within the exact same block, the slippage protection applies linearly over the current state of the pool. An MEV attacker can maliciously skew the AMM price prior to the swap, and the protocol will incorrectly fetch the manipulated spot price as its baseline quote, allowing slippage checks to pass on the distorted metrics.

Furthermore, when calling the Uniswap Universal Router to execute the swaps (`swapForOutput` and `swapForInput` inside `UniswapV3SwapperEngine.sol`), the deadline parameter is hardcoded to `block.timestamp`. This means the deadline is dynamically set to whatever the current block timestamp is when the transaction is finally included by a validator, not when it was originally submitted. Without an absolute future deadline, transactions can sit in the mempool indefinitely during network congestion, remaining exposed to extreme market volatility and delayed MEV targeting.

Impact

Immediate and recurring loss of protocol funds. The combination of same-block spot pricing and unbounded execution deadlines makes every high-value protocol-initiated swap highly susceptible to sandwich attacks and volatile market conditions. The protocol continually absorbs the inflated exchange rates, directly depleting staked backing pools and operator margins.

Recommendation

To prevent same-block AMM manipulation, do not use instantaneous spot quotes to derive slippage limits. Instead, securely compute slippage protection boundaries using a Time-Weighted Average Price (TWAP) from the Uniswap pool or a reputable decentralized price oracle such as a Chainlink Data Feed. Additionally, implement an explicit deadline parameter (e.g., an absolute future timestamp like `block.timestamp + maxDelay`) for `swapForOutput` and `swapForInput` to enforce a strict expiration window.

Note

Spice team informed Hashlock that the use of spot pricing from the AMM is part of the intended design due to the protocol's reliance on AMM non-linear pricing. The system performs a unit-based pricing validation before executing swaps to ensure paths with adequate liquidity are selected. This mechanism is combined with a pricing oracle and additional slippage protection through the `swapForInputQuote` and `swapForOutputQuote` methods to improve pricing reliability for large swaps.

Status

Acknowledged

[M-02] EigenCoverageProviderFacet#issueClaim - Strategy Removal Can Block Claim Issuance

Description

Claim issuance allows a coverage agent to create a new claim against an active position, lock reward funds, and adjust operator coverage exposure. This flow is expected to remain available for valid positions until expiry.

Vulnerability Details

The `issueClaim` function calls `_validatePosition`, which enforces that `assetToStrategy[data.asset]` is still present in `_strategyWhitelist`. If governance removes a strategy from the whitelist after positions have already been created, `_validatePosition` reverts with `StrategyNotWhitelisted`.

```
function _validatePosition(CoveragePosition memory data, uint256 amount, uint256
duration, uint256 reward)

    private

    view

    {

        uint256 minimumReward = (amount * data.minRate * duration) / (10000 * 365 days);

        if (minimumReward > reward) revert InsufficientReward(minimumReward, reward);

        if (!_strategyWhitelist.contains(assetToStrategy[data.asset])) {

                                                                revert
IEigenOperatorProxy.StrategyNotWhitelisted(assetToStrategy[data.asset]);

        }

        if (data.maxDuration > 0 && duration > data.maxDuration) revert
DurationExceedsMax(data.maxDuration, duration);

        require(

            duration + block.timestamp <= data.expiryTimestamp,
```



```
        DurationExceedsExpiry(data.expiryTimestamp, duration + block.timestamp)  
  
    );  
  
}
```

The root cause is that claim issuance for existing positions is coupled to a mutable whitelist check, even though the position was opened under previously valid conditions.

Impact

Coverage agents may be unable to issue claims for still-active positions if the associated strategy is removed from the whitelist. This creates a denial-of-service condition for legitimate claims and introduces governance-dependent liveness risk.

Recommendation

We recommend decoupling claim issuance for existing positions from dynamic whitelist checks, or snapshotting strategy validity at position creation to ensure claims remain issuable for the lifetime of the position.

Status

Resolved

[M-03] AssetPriceOracleAndSwapper#swapForInput - swapForInput Passes Parameters in Reversed Order, Destroying Slippage Protection

Description

The `swapForInput` function in `AssetPriceOracleAndSwapper.sol` performs an exact-input swap using a minimum output amount derived from `swapForInputQuote`. However, it passes parameters to `swapForInputQuote` in reversed order, causing the slippage calculation to use incorrect exchange rates and token decimals.

Vulnerability Details

In the `swapForInput` function, the `assetA` and `assetB` parameters are swapped when calling `swapForInputQuote`. The function signature expects `(amountIn, assetA, assetB)`, where `assetA` is the output token, and `assetB` is the input token, but `swapForInput` passes `(amountIn, assetB, assetA)`.

```
uint256 minAmountOut = swapForInputQuote(amountIn, assetB, assetA);

//                      ^^^^^^  ^^^^^^  SWAPPED
```

This causes the quote calculation to interpret the input amount as being denominated in the wrong token, leading to catastrophically incorrect minimum output bounds.

Impact

The bug manifests differently depending on swap direction:

Case 1: Zero Slippage Protection

For swaps where the reversed calculation produces a tiny minimum output (e.g., `swapForInput(3000e6, WETH, USDC)` → ~9 wei minimum), the swap proceeds with essentially no slippage protection, enabling full MEV extraction.

Case 2: Permanent DoS

For swaps in the opposite direction (e.g., `swapForInput(1e18, USDC, WETH)`), the reversed calculation demands an impossibly large minimum output (~333 million ETH), causing all swaps to revert.

The `swapForInput` function is exposed in the diamond's public interface and can be called by any integrated component, making this a system-wide vulnerability affecting all exact-input swaps.

Recommendation

Fix the parameter order in `swapForInput`:

```
// Before (broken):  
  
uint256 minAmountOut = swapForInputQuote(amountIn, assetB, assetA);  
  
// After (fixed):  
  
uint256 minAmountOut = swapForInputQuote(amountIn, assetA, assetB);
```

Status

Resolved

[M-04] AssetPriceOracleAndSwapper#SwapForOutput - SwapForOutput Uses Reversed Exact-In Quote and Underestimates Max Input

Description

The `swapForOutput` flow in `AssetPriceOracleAndSwapper.sol` must compute a safe `amountInMax` for an exact-output swap. Instead of using an exact-output quote, it calls `swapForOutputQuote` with reversed assets, which then uses `ISwapperEngine.getQuote(...)` (an exact-input style quote).

This means the system derives an exact-output bound from a reverse-direction exact-input simulation, which applies pool fees in the wrong mathematical direction.

Vulnerability Details

In `swapForOutput(...)`, arguments are unintentionally flipped when building the quote:

```
function swapForOutput(uint256 amountOut, address assetA, address assetB) public {  
    AssetPair memory _assetPair = _getRegisteredAssetPair(assetA, assetB);  
  
    uint256 maxAmountIn = swapForOutputQuote(amountOut, assetB, assetA);  
  
    // ...  
}
```

`swapForOutputQuote(...)` then calls:

```
maxAmountIn = ISwapperEngine(_assetPair.swapEngine).getQuote(  
    _assetPair.poolInfo,  
    amountOut,  
    assetA,  
    assetB  
);
```

But `ISwapperEngine.getQuote(...)` is an exact-input quote primitive:

```
uint256 maxAmountIn = swapForOutputQuote(amountOut, assetB, assetA);

// ...

}
```

So the protocol asks: "If I input `amountOut` of output token, how much input token do I get back?" and reuses that value as `amountInMax` for an exact-output swap.

For a single pool with fee f , desired output Y , and spot conversion Y/price :

- The correct exact-output input requirement is proportional to $Y / (1 - f)$.
- Reversed exact-input quote returns proportional to $Y * (1 - f)$.

Therefore, the estimated max input is short by a factor of $(1 - f)^2$ versus the required exact-output input, before slippage is applied.

Impact

Exact-output swaps can revert even when market conditions are normal, because `amountInMax` is systematically too low. This can cause denial of service for protocol flows that rely on exact-output settlement (for example, fixed debt repayment or fixed-size position adjustments), especially when configured slippage is tighter than the structural quote error.

Recommendation

Add a native exact-output quote interface and use it for `swapForOutputQuote(...)`.

1. Extend `ISwapperEngine` with an exact-output quote function.
2. Implement it in `UniswapV3SwapperEngine` using Uniswap's exact-output quote path.
3. In `swapForOutputQuote(...)`, keep forward argument ordering and call the new exact-output quote API.

Example interface addition:

```
function getQuoteForOutput(bytes memory poolInfo, uint256 amountOut, address base,
address quote)

external
```

```
view
```

```
returns (uint256 amountIn);
```

Note

Spice team informed Hashlock that the quoting mechanism is designed around unit-based pricing to validate swap conditions before execution. This approach helps ensure that swap paths with sufficient liquidity are selected and aims to prevent mispricing during large swaps. The protocol also relies on a pricing oracle and slippage protection mechanisms to further safeguard swap execution.

Status

Acknowledged

[M-05] EigenServiceManagerFacet#registerOperator - registerOperator Has Inverted Caller Check

Description

The `registerOperator(...)` callback in `EigenServiceManagerFacet` is intended to be invoked by EigenLayer's `AllocationManager` during operator set registration. However, the implementation validates the wrong caller and uses an inverted comparison, allowing unauthorized callers through.

Vulnerability Details

The comparison uses `!=` instead of `==` and targets the wrong contract. This means:

- If `msg.sender == delegationManager`, it reverts.
- If `msg.sender != delegationManager` (which includes literally anyone), it passes.

```
function registerOperator(  
    address operator, address _avs, uint32[] calldata, bytes calldata  
) external {  
    require(msg.sender != _eigenAddresses.delegationManager, "Not delegation manager");  
  
    // ...  
}
```

Impact

Attackers can repeatedly reset an operator's custom `coverageThreshold` to the default value (7000), overriding operator risk configuration and causing threshold griefing/state tampering.

This issue does not, by itself, complete EigenLayer registration state in `AllocationManager`; the direct impact is unauthorized mutation of AVS-local operator threshold state.

Recommendation

Enforce strict caller validation against `allocationManager`:

```
require(msg.sender == _eigenAddresses.allocationManager, "Not allocation manager");
```

Also, update tests to assert:

- non-allocationManager callers revert,
- delegationManager reverts,
- allocationManager succeeds.

Status

Resolved

[M-06] EigenCoverageProviderFacet#closePosition - Whitelist Removal Bypasses Operator Permission Check

Vulnerability Details

The `closePosition` function allows a user to close an active coverage position before expiry. If the associated strategy is still whitelisted, the function additionally verifies operator permissions through `_checkOperatorPermissions` before finalizing closure.

Vulnerability Details

The permission check is wrapped in the condition `_strategyWhitelist.contains(strategy) && !_checkOperatorPermissions(...)`. Due to short-circuit evaluation, if the strategy has been removed from `_strategyWhitelist`, the left side evaluates to false, and `_checkOperatorPermissions` is never executed. The root cause is that operator authorization is conditionally enforced based on dynamic whitelist membership, allowing the check to be skipped entirely when a strategy is removed.

```
function closePosition(uint256 positionId) external {
    CoveragePosition storage positionData = positions[positionId];

    address operator = address(uint160(uint256(positionData.operatorId)));
    address strategy = assetToStrategy[positionData.asset];

    require(
        positionData.expiryTimestamp >= block.timestamp, PositionExpired(positionId,
positionData.expiryTimestamp)
    );

    positions[positionId].expiryTimestamp = block.timestamp;

    if (
        _strategyWhitelist.contains(strategy)
        && !_checkOperatorPermissions(
```



```
operator, _eigenAddresses.allocationManager,  
IAallocationManager.modifyAllocations.selector  
    )  
    ) revert IEigenServiceManager.NotOperatorAuthorized(operator, msg.sender);  
  
    emit PositionClosed(positionId);  
}
```

Impact

If the strategy is removed from the whitelist, position holders can close positions without any operator permission validation. While this may be acceptable by design, it creates inconsistent authorization behavior depending on mutable whitelist state.

Recommendation

We recommend enforcing operator permission checks independently of whitelist membership, or explicitly documenting that permission validation is intentionally skipped when a strategy is removed.

Status

Resolved

[M-07] EigenCoverageProviderFacet#closeClaim - Rewards Locked for Slashed Refundable.Full Claims

Description

Refundable.Full rewards are only released when a claim reaches Completed. Slashing moves a claim to Slashed, and there is no transition from Slashed or Repaid to Completed. This leaves the reward held by the Diamond with no release path.

Vulnerability Details

- `closeClaim(...)` only accepts Issued or Reserved, so a Slashed claim cannot be completed.
- `captureRewards(...)` only releases rewards for Refundable.Full when the status is Completed.
- `repaySlashedClaim(...)` can move Slashed → Repaid, but does not complete the claim or release the reward.

Impact

Rewards for Refundable.Full claims that get slashed cannot be released to either party. The funds remain stuck in the Diamond contract.

Recommendation

Define a terminal path for Refundable.Full rewards when a claim is slashed or repaid. For example:

- transition Slashed (or Repaid) to Completed when appropriate, so `captureRewards(...)` can release the reward, or
- Add a dedicated reward-release flow for slashed Refundable.Full claims (refund to the coverage agent or distributed to the operator).

Status

Resolved

Low

[L-01] UniswapV3SwapperEngine#swapForOutput, swapForInput - Deadline is set to `block.timestamp`

Description

In the `swapForOutput` and `swapForInput` functions, when calling the router to swap tokens, the deadline parameter is set to `block.timestamp`. This means the deadline is set to the current block timestamp when the transaction is included by the validator, not when it's submitted. Without a deadline parameter, the transaction may sit in the mempool and be executed at a much later time, potentially resulting in a worse price for the user.

However, due to the `amountIn` being a small amount, the impact is low.

Recommendation

Consider adding a deadline parameter to the `swapForOutput` and `swapForInput` functions, allowing an appropriate future timestamp.

Status

Resolved

[L-02] EigenCoverageProviderFacet#_validatePosition - Zero-Duration TimeWeighted Claims Revert in captureRewards

Description

`_validatePosition(...)` does not enforce `duration > 0`, so a claim can be issued (or converted) with `duration = 0`. For `Refundable.TimeWeighted`, `captureRewards(...)` then executes a division by `_claim.duration`, which causes a division-by-zero panic.

Recommendation

Enforce `duration > 0` in claim validation paths (issuance and reserved conversion). Add a defensive guard in `captureRewards(...)` for legacy zero-duration claims to avoid hard reverts.

Status

Resolved.

QA

[Q-01] EigenCoverageProviderFacet#liquidateClaim - liquidateClaim Comment Mismatch on createdAt Reset

Description

Comment states `createdAt` is reset to `block.timestamp` when reassigning a claim; `liquidateClaim` only updates `positionId`.

```
// Update the claim to the new position and reset the createdAt to the current block timestamp

_claim.positionId = positionId;
```

Impact

The function reverts to expired claims (`ClaimExpired`), so the mismatch is documentation-only.

Recommendation

Align the comment with the implementation:

```
// Update the claim to the new position (createdAt is preserved to maintain original expiry timeline)

_claim.positionId = positionId;
```

Status

Resolved

[Q-02] EigenCoverageProviderFacet#onIsRegistered

-

coverageAgentToOperatorSetId is write-once

Description

coverageAgentToOperatorSetId is effectively write-once. After the first registration, a coverage agent address cannot be reassigned to a new operator set.

In `EigenCoverageProviderFacet.onIsRegistered()`, the contract reverts if `coverageAgentToOperatorSetId[msg.sender] != 0`, then sets it once.

There is no lifecycle function to clear or rotate this mapping (no deregistration/reset/migration path).

Impact

A registered coverage agent address cannot move to a different operator set even if governance or operations require it.

Recommendation

Add a deregistration path.

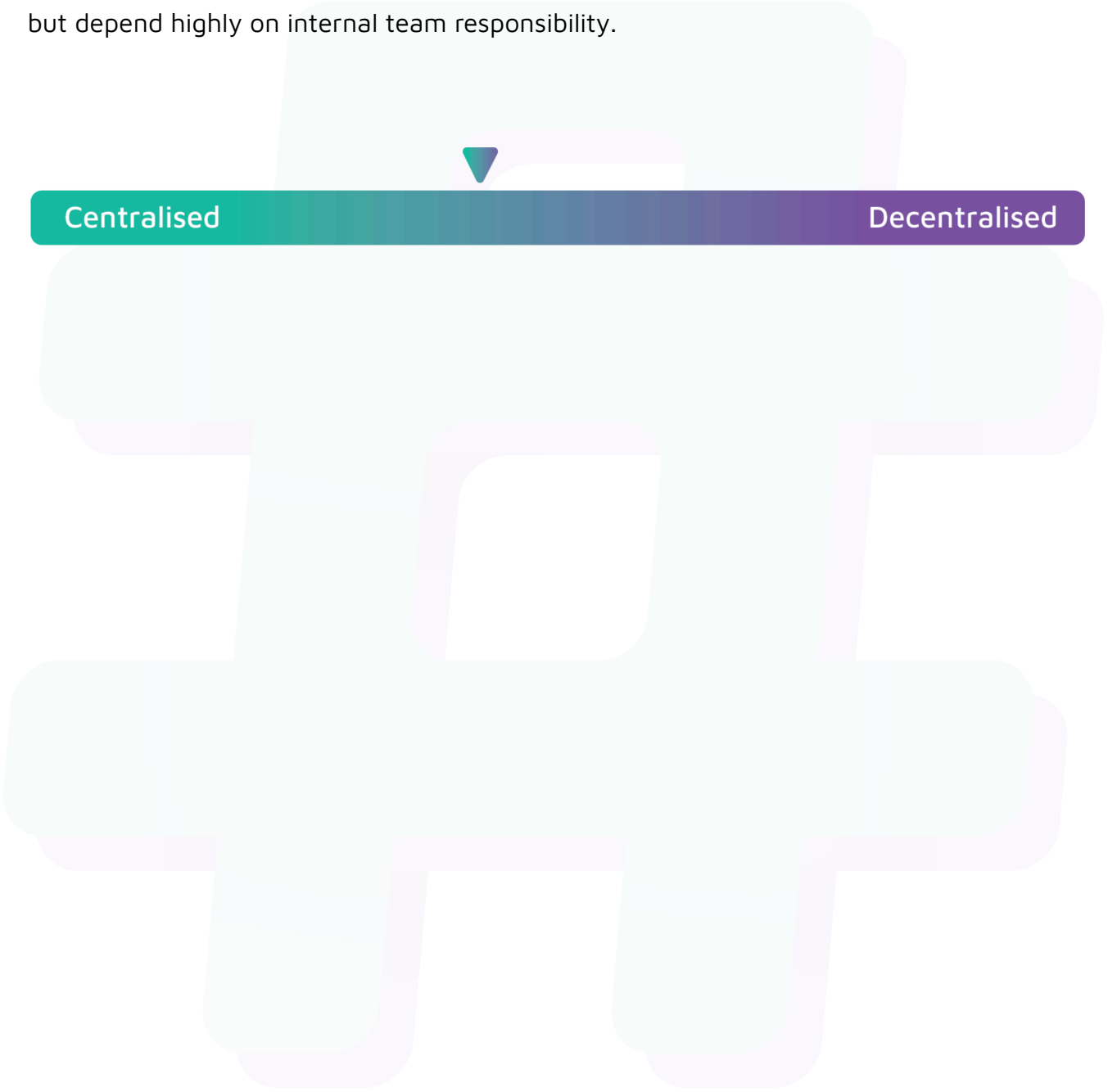
Status

Acknowledged.

Centralisation

The Spice project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, the Spice project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.

